Vibecode and Deploy a Frontend for an ADK agent

About this codelab

*subject*Last updated Jun 18, 2026
*account_circle*Written by Sita Lakshmi Sangameswaran

1. Introduction

This lab is part of Kaggle's **5-Day AI Agents: Intensive Vibe Coding Course with Google**. You can find the additional codelabs and resources available on the event site.

Prerequisite: This lab assumes you have already deployed the ambient expense agent to Agent Runtime. If you have not completed this step, please finish the prerequisite codelab first: Deploy an ADK agent to Agent Runtime using Agents CLI.

In the previous lab, you **vibecoded** an ambient expense agent and **deployed it to Agent Runtime on Google Cloud**. While your agent is now live in the cloud, interacting with it requires making direct API requests or issuing prompts from the Google Cloud Console.

In this codelab, you will give your agent a **fully functional front door and a human-in-the-loop management dashboard**. Acting as the **software architect**, you will guide **Antigravity** (Google's agentic IDE) to vibecode a web-based **Manager Dashboard**, deploy it to **Cloud Run**, and integrate it with an **asynchronous, event-driven architecture** powered by **Pub/Sub**.

What you'll build

Here is the high-level event-driven topology you will build:

1 1 **Event Ingestion:** Expense payloads are published to Pub/Sub and pushed directly to the Agent Runtime.
2 2 **Auto-Approval:** Low-value expenses (< $100) are processed and approved instantly.
3 3 **Human-in-the-Loop:** High-value expenses (>= $100) pause execution and persist state in the Session Service.
4 4 **Manager Resolution:** The Cloud Run dashboard surfaces paused sessions, allowing managers to click Approve or Reject to resume agent execution.

What you'll need

- • A Google Cloud project with billing enabled.
- • The **deployed agent** from the previous lab (its remote Agent Runtime ID) and the Google Cloud project it runs in.
- • A terminal with **gcloud** (a command line tool for Google Cloud), **Python**

3.11+, and **uv**.

- **Antigravity** installed. See the official website.

2. Reconnect Antigravity and confirm deployment

Open your existing project folder in **Antigravity**. This lab picks up exactly where the previous deployment lab ended, so you should already have the agent running on Agent Runtime. In this step, you will guide Antigravity through three prompts to ensure your environment is fully prepared.

Note for Antigravity users: During this codelab, Antigravity may generate implementation plans or display popups before executing commands or writing code. Be sure to review and **approve** these plans or popups to allow Antigravity to proceed with the tasks.

Quota Tip: If you run out of quota during testing or development, switch the model in Antigravity to another available model (such as `gemini-2.5-pro` or `gemini-3-flash`).

1. Verify ADK skills

First, ensure Antigravity has loaded the correct ADK skills.

👉 **Prompt to Antigravity:**

```
Reload your adk-scaffold skill and verify that the required
ADK skills for this lab are active.
```

What to expect: Antigravity will confirm that the necessary ADK skills are active in your workspace, ensuring it is ready to interact with ADK session services and structures.

2. Configure Google Cloud environment

Next, connect Antigravity to your Google Cloud project and enable the required service APIs.

👉 **Prompt to Antigravity:**

```
Help me set up my Google Cloud environment. Connect to my
project `YOUR_PROJECT_ID`
in the global region, authenticate, and enable the
necessary generative platform APIs
(aiplatform.googleapis.com, run.googleapis.com,
pubsub.googleapis.com, cloudbuild.googleapis.com).
```

What to expect: Antigravity will execute `gcloud` commands to set your active project, verify your authentication credentials, and ensure that Agent Platform, Cloud Run, Pub/Sub, and Cloud Build APIs are enabled.

3. Confirm deployed agent and align on objectives

Finally, point Antigravity to your existing live agent and establish the architecture goals for this lab.

👉 **Prompt to Antigravity:**

```
Get the already running expense agent from Agent Runtime
by checking the deployment metadata in this project. We are
NOT changing the agent's code
in this lab. We are building a Pub/Sub event pipeline and a
Manager Dashboard in front of it.
Wait for more instructions before proceeding.
```

What to expect: Antigravity will inspect your local `deployment_metadata.json` file to locate the remote Agent Runtime ID, acknowledge that the agent code remains untouched, and confirm it is ready to begin building the event pipeline and dashboard.

3. Vibecode a frontend dashboard for the Expense Agent

With your cloud environment configured and agent verified, you now need a mechanism for managers to interact with paused agent sessions and approve expenses. When an expense report exceeds the $100 threshold, the ambient expense agent automatically halts execution at a human-in-the-loop `RequestInput` node and preserves its state within the Agent Platform Session Service.

To make these paused sessions actionable, you will guide Antigravity to vibe-code a standalone FastAPI web application. FastAPI is a popular web framework for building APIs with Python. This service acts as the bridge: it dynamically queries the session service for pending approvals, presents them in an elegant interactive web UI, and provides endpoints to securely resume the agent's execution on Agent Runtime once a decision is made.